STAR: Structure-aware Typed Adaptive Retrieval

A smart knowledge base for agents over high-stakes financial documents

CognitionNext '26 • Problem Statement 1 • Interim technical report (work in progress)

Summary. We treat retrieval as *choosing a calibrated evidence set*, not ranking a top-k list. STAR splits documents into paragraph-sized typed units and builds a catalogue with a *version-family lattice*, so near-duplicate documents become structure rather than noise. It then scores every candidate paragraph with a learned, query-relative model, propagates evidence over a paragraph graph, and returns the set that maximises expected precision and recall for that particular question. Selection uses no LLM calls; each query makes exactly one LLM call, to write the answer. On the training set (all 100 questions, held-out folds, free text-overlap proxy) STAR reaches retrieval F1 0.31 against 0.15 for the banned hybrid-search baseline, and recall 0.67 against 0.25. These are early numbers. Section 6 lists the known gaps and the next fixes.

Contents

1 The problem and how it is scored **1**

2 What the training data shows **2**

2.1 Corpus and questions 2

2.2 Relevance clusters along the document 2

2.3 The judge's behaviour 2

3 The STAR architecture **3**

4 Comparison architectures **5**

5 Results so far **5**

6 Known gaps and next steps **6**

1 The problem and how it is scored

Given a question and an unorganised dump of PDFs, the system must return a grounded answer, the paragraphs it used (with document and page), and a step-by-step trace. It is scored on four LLM-judged metrics plus cost and latency. We read the organisers' evaluation notebook closely, because the exact scoring shapes the design.

Metric	How the official evaluator computes it (judge: `deepseek-v4.1-flash`, temperature 0)
Answer precision	Both answers are split into atomic points. $1 - \frac{\#\text{our points not supported by gold}}{\#\text{our points}}$
Answer recall	$\frac{\#\text{gold points covered by our answer}}{\#\text{gold points}}$
Context precision	One judge call sees all returned chunks: $\frac{\#\text{judged relevant}}{\#\text{returned}}$
Context recall	$\frac{\#\text{judged relevant}}{\#\textbf{gold chunks}}$. *Count-based and not capped at 1.*

Table 1: Scoring definitions from the official notebook.

Three consequences drive the whole design.

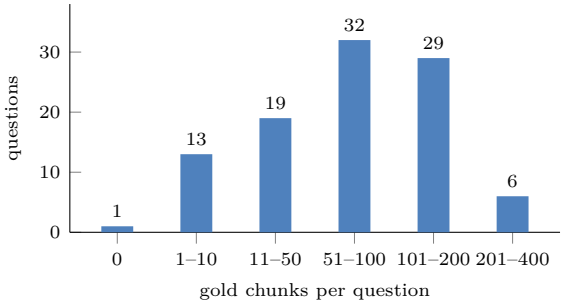
1. **Granularity is set by the recall denominator.** The median question has 81 gold chunks. Ten big, correct chunks cap recall at about 0.12, so evidence units must be paragraph-sized and *how many* we return must change per question.
2. **Any point not in the gold answer lowers precision**, even if it is true. Answers must use only retrieved evidence.
3. **The judge breaks on long answers** (Section 2.3), so answers must stay under about 700 words.

2 What the training data shows

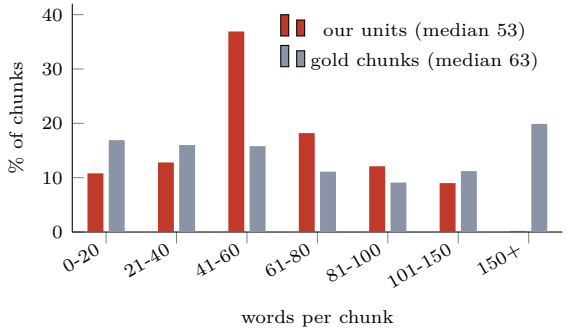
2.1 Corpus and questions

Corpus		Questions (100)	
PDFs	35	A: credit analysis of annual reports	36
Pages (parsed)	6,970	B: Basel vs US capital proposals	32
Scanned pages needing OCR	97	C: CECL accounting & quant models	32
Largest document (distractor)	1,982 pp	Gold documents per question	0–9
Paragraph units produced	46,304	Gold chunks per question (median)	81

Table 2: Training set. The 1,982-page consolidated Basel Framework is never cited in gold: it is a near-duplicate distractor.



(a) Gold evidence size varies by two orders of magnitude, so a fixed k fails in both directions.



(b) Our paragraph units are sized like the gold chunks. Gold has a long tail of large tables.

Figure 1: Evidence size statistics.

The gold chunks were clearly made by a vision-language document parser. Their section names split into prose (4,838), markdown tables (1,056), LaTeX formulas (277) and image descriptions (68). So table handling matters a lot, and a small number of facts exist only as images.

2.2 Relevance clusters along the document

Gold evidence for one question and document spans a median of 9 pages, in about 5 separate runs. We mapped gold chunks onto our units by word 4-gram overlap and measured *homophily*: how much more likely a unit is to be relevant when its graph neighbour is.

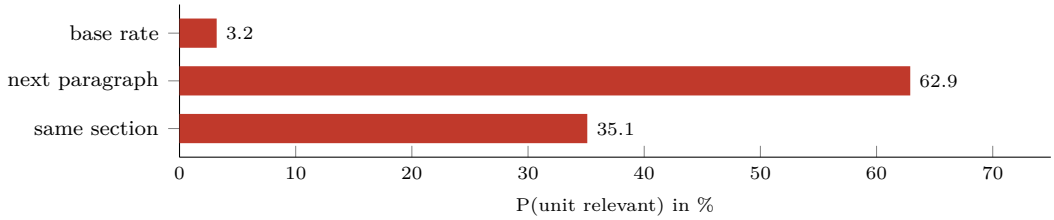


Figure 2: If a paragraph is relevant, the next paragraph is relevant 62.9% of the time, **19× the 3.2% base rate**. A paragraph in the same section is relevant 11× as often. This is the empirical case for a graph layer.

2.3 The judge's behaviour

- In our first oracle run (answers written from the gold chunks), answers ran 1,000–4,600 words and **the official judge failed on 18 of 24 questions**. STAR therefore caps answers at about 700 words, near the gold median of 754.
- **The judge rejects about 25% of gold chunks** as irrelevant, so returning exactly the gold chunks scores only about 0.75 on context metrics. The selector should learn *what the judge accepts*, not only what gold contains.

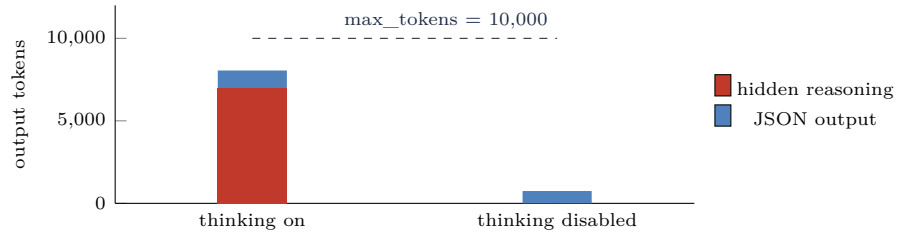


Figure 3: Tokens the judge used to split one 654-word gold answer into points. With the notebook's settings, reasoning takes 87% of the budget. Answers much longer than this exhaust it, the call returns no content, and the question cannot be scored.

- One question (A36) is a version trap: it asks about FY2025, but the corpus holds only the 2024 report, and the judge rejected every chunk. Version awareness is enforced by the judge itself.

3 The STAR architecture

STAR has six layers. LLM calls happen only once per *document* at ingestion (the catalogue card) and once per *query* (the answer). Everything between runs locally on CPU in milliseconds.

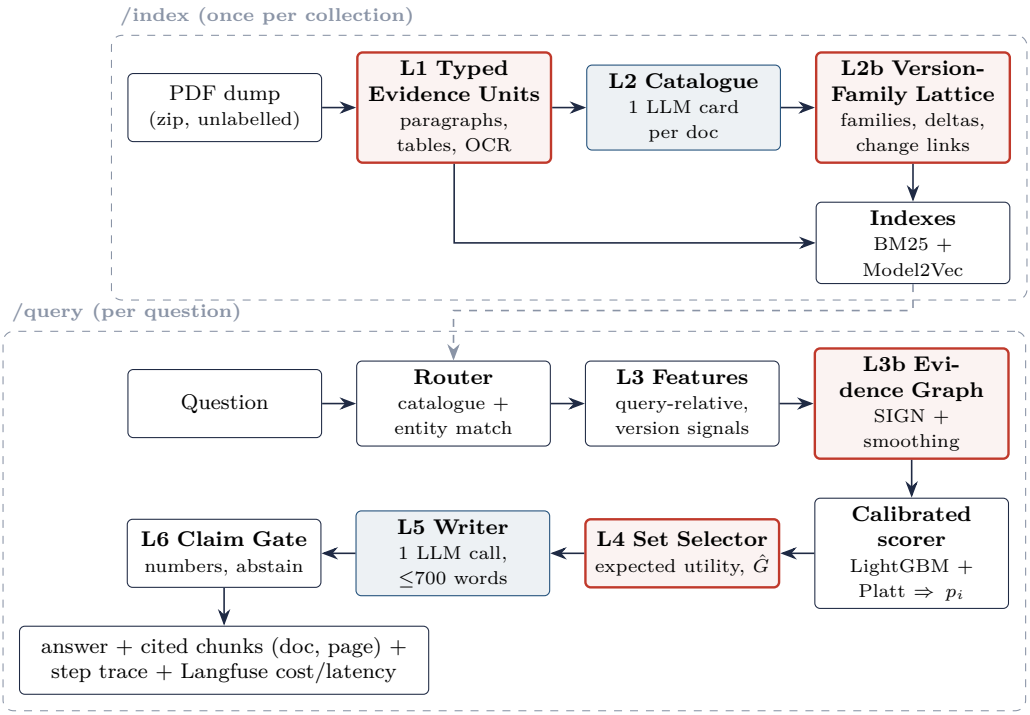


Figure 4: STAR pipeline. Red boxes are our main contributions; blue boxes are the only LLM calls.

L1 Typed Evidence Units (fixes the chunk-size dilemma)

Each page is extracted with PyMuPDF. Pages with no text layer go through Tesseract OCR. Running headers and footers are removed, and number-heavy pages pass through table detection, so each table becomes one markdown unit. Prose blocks are merged or split to 40–150 words, with headings tracked as a section path. *Completeness comes from how many units we select, not from making units bigger.* The training corpus (6,970 pages) ingests in about 6 minutes on a laptop CPU.

L2 Catalogue and L2b Version-Family Lattice (fixes near-duplicates)

One cheap LLM call per document fills a typed card: title, issuer, entities, document type, jurisdiction, period, date and topics. The 35 cards cost $0.025 in total. The lattice then groups documents into version families.

It uses text containment between documents ($|A \cap B|/|A|$ over word 5-gram shingles) and joins pairs with union-find.

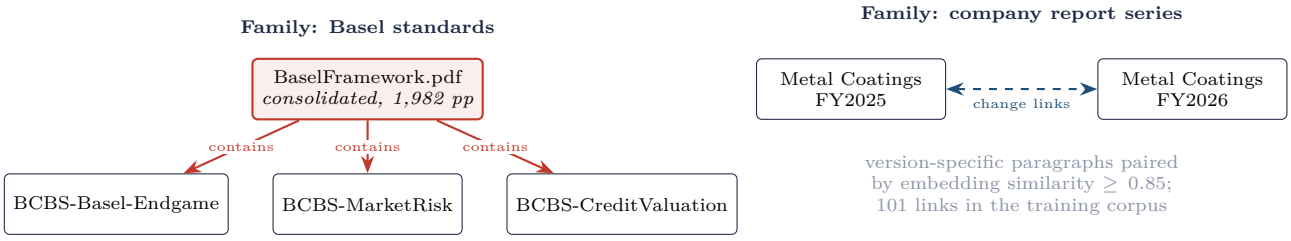


Figure 5: Two of the five version families found automatically. The consolidated Basel Framework, the distractor, is flagged because it contains most of its smaller siblings.

Each unit is tagged *shared* (its text exists in another version) or *delta*. The training corpus has 4,534 shared and 41,770 delta units. Query intent is classified by rules: comparison words, or two or more years, mean *compare*; anything else is *single*. Intent becomes soft features: "compare × has family" and "single × wrong period". This follows the idea of VersionRAG [1], reimplemented in our code, with two differences: it uses no LLM per query, and it detects consolidated texts.

L3 Query-relative features and calibrated scoring

The router scores documents from card BM25, the top paragraph similarities and entity-name hits, and keeps the top 10 documents as a soft candidate pool. Every paragraph in the pool gets 25 features: dense and BM25 scores as raw values, z-scores and percentiles *within the pool*; document route score; neighbour and section maxima; table flag; number overlap with the query; and the lattice signals. Because every feature is relative to the query, the model transfers to a corpus it has never seen. A LightGBM classifier is trained with 5 folds grouped by question, and Platt scaling turns its scores into probabilities p_i.

L3b Evidence Graph (uses the 19× homophily)

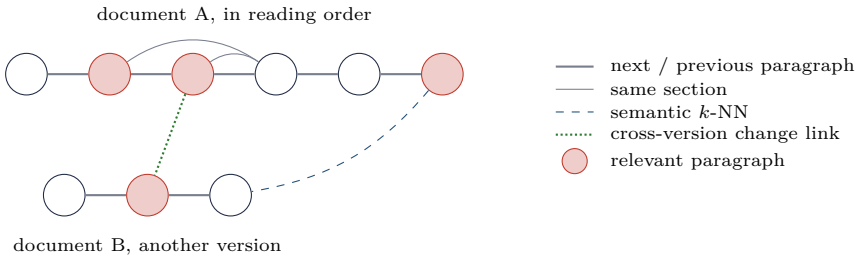


Figure 6: The evidence graph over one question's candidate pool. Relevant paragraphs come in runs; edges let one strong paragraph lift its weaker neighbours.

We use two graph-learning designs that work with little labelled data, instead of training a deep GNN on 100 questions:

- **SIGN / SGC** [2, 3]: precompute diffused features $[X,\ \hat{A}X,\ \hat{A}^2X]$ with $\hat{A} = D^{-1/2}(A+I)D^{-1/2}$, then train LightGBM on them. This is message passing without backpropagation through the graph.
- **Correct & Smooth** [4]: smooth the predicted probabilities over the graph, $z \leftarrow (1-\alpha)p + \alpha\hat{A}z$.

L4 Expected-Utility Set Selector (decides how much to return)

Assuming relevance labels are independent given p_i, the utility-optimal set is a prefix of units sorted by p_i [5]. For a prefix of size k with $S_k = \sum_{i \le k} p_{(i)}$ and estimated gold count $\hat{G} = \sum_i p_i$:

$$\mathbb{E}[\text{precision}_k] \approx \frac{S_k}{k}, \qquad \mathbb{E}[\text{recall}_k] \approx \min\left(1,\ \frac{S_k}{\hat{G}}\right), \qquad k^\star = \arg\max_k\ \mathrm{F1}\big(\mathbb{E}[P_k], \mathbb{E}[R_k]\big).$$

A factual lookup gets a handful of units, and a full credit review gets hundreds. Capping recall at 1 means the objective never rewards splitting text into fragments to inflate the count.

L5 Writer and L6 Claim Gate

One `deepseek-v4.1-flash` call with thinking disabled writes the answer from the selected units, in document order and in the gold answers' analyst style (cited `[file pN]`), using no outside knowledge, in at most about 700 words. The gate then checks that every number and entity appears in a cited unit, and returns the exact abstention string when even the best p_i is low.

4 Comparison architectures

Row	Design	LLM calls / query	Status
STAR	This report: typed units, lattice, calibrated graph scorer, expected-utility set	1 (writer)	measured (proxy)
1	Banned baseline: hybrid dense + BM25 (reciprocal rank fusion), fixed top-k	1	measured (proxy)
2	LLM-Navigator after the Enterprise RAG Challenge winner [6]: the LLM reads shortlisted pages and picks paragraphs	1 + pages/6	built, not run
3	Brute-force long context: the routed documents in one prompt (the PS author's upper bound)	1 (very long)	planned

Table 3: Architectures compared. Rows 2 and 3 anchor the expensive end of the cost-versus-accuracy chart.

5 Results so far

All retrieval numbers use the **free text-overlap proxy**: a selected unit counts as correct if most of its word 4-grams appear in a gold chunk, and recall divides by the gold-chunk count, as the official metric does. They come from all 100 questions, with every learned model evaluated out-of-fold (5 folds grouped by question).

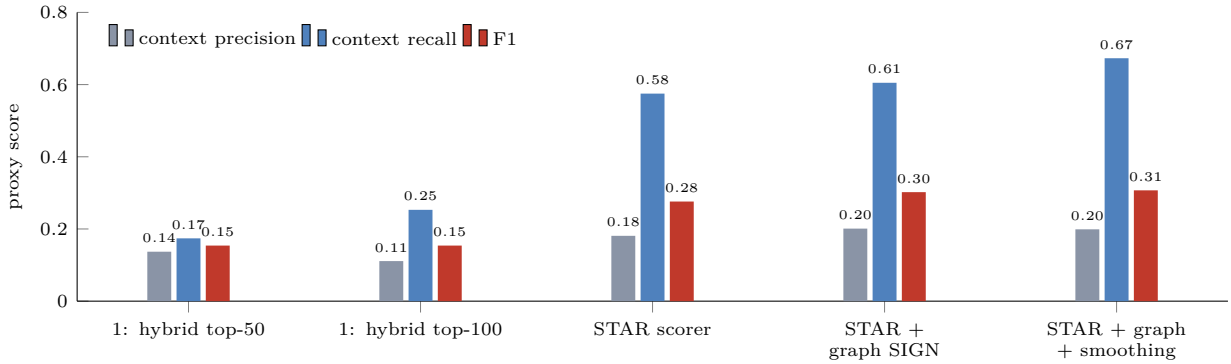


Figure 7: Retrieval quality on the proxy. STAR's set selection more than doubles recall over fixed top-k, and the graph layer adds recall on top.

Configuration	P	R	F1	median k	$\rho(\hat{G}, \lvert\text{gold}\rvert)$
1: hybrid RRF, top-20	0.177	0.097	0.126	20	–
1: hybrid RRF, top-100	0.111	0.253	0.154	100	–
LightGBM scores, fixed top-100	0.196	0.386	0.260	100	–
STAR scorer + set selector	0.181	0.575	0.276	186	0.27
+ evidence graph (SIGN)	**0.201**	0.605	0.302	172	0.31
+ Correct & Smooth ($\alpha = 0.5$)	0.199	**0.673**	**0.307**	207	0.31
ablation: smoothing without SIGN	0.180	0.656	0.282	200	0.27

Table 4: Ablation. Each graph component adds measurable recall; SIGN features also add precision.

Routing. 93.8% of gold documents land in the candidate pool, and 87.7% of the gold count is reachable inside it. That is the retrieval ceiling for the current router.

Cost so far. The whole development run, including judge calls, has spent $2.11 of the $10 credit, most of it on judge calls that failed for the reason shown in Figure 3. STAR's query path runs no LLM calls until the writer.

6 Known gaps and next steps

1. **Proxy precision understates the real score.** The proxy only credits paragraphs that contain gold text, while the judge also accepts relevant paragraphs outside gold (and rejects 25% of gold). Next: run the judge's relevance check on STAR's selections for the 24-question subset (about $0.30) to calibrate proxy against judge.
2. **The set-size estimate $\hat{G}$ is weak** (correlation 0.31 with the true gold count), so STAR over-selects (median 172–207 units). Next: a small count model from query features, which directly raises precision.
3. **Sharper relevance:** add a CPU cross-encoder score on the top few hundred candidates as an extra feature (confirmed allowed), and tighten the 10-document pool.
4. **Run rows 2 and 3** on the subset to anchor the cost-versus-accuracy chart.
5. **End-to-end answers:** run the writer on STAR's selections and score them with the official judge; serve `/index` and `/query` with the trace and Langfuse; export the knowledge base as an Open Knowledge Format bundle.

References

[1] D. Huwiler et al. *VersionRAG: Version-aware retrieval-augmented generation.* arXiv:2510.08109, 2025.

[2] E. Rossi et al. *SIGN: Scalable Inception Graph Neural Networks.* arXiv:2004.11198, 2020.

[3] F. Wu et al. *Simplifying Graph Convolutional Networks.* ICML 2019.

[4] Q. Huang et al. *Combining Label Propagation and Simple Models Out-performs Graph Neural Networks.* ICLR 2021.

[5] N. Ye, K. Chai, W. Lee, H. Chieu. *Optimizing F-measures: A Tale of Two Approaches.* ICML 2012.

[6] I. Rice. *RAG-Challenge-2*, winner of the Enterprise RAG Challenge round 2. github.com/IlyaRice/RAG-Challenge-2 (MIT), 2025.